Part 1 — Website Visit Analysis (TXT)

Task 1

Read website_visits.txt .

```
with open("website_vists.txt") as f:
    visits=[line.strip() for line in f]
```

Task 2

Print all visitors.

```
print(visits)
```

Task 3

Find the total number of visits.

```
print(len(visits))
```

Task 4

Find unique visitors using a set.

```
unique_visitors=set(visits)
print(unique_visitors)
```

Task 5

Count how many times each visitor came to the website.
Example expected structure:

```
{
"Rahul":3,
"Sneha":3,
"Arjun":2,
"Priya":1,
"Karan":1
}
```

```
visit_count={}
for v in visits:
    visit_count[v]=visit_count.get(v, default: 0)+1
print(visit_count)
```

Task 6

Find the most frequent visitor.

```
most_visitor=max(visit_count,key=visit_count.get)
print(most_visitor)
```

Part 2 — Product Catalog Analysis (JSON)

Task 7

Read products.json .

```
import json
with open("products.json") as f:
    products_data=json.load(f)["products"]
```

Task 8

Print all product names and prices.

```
for p in products_data:
    print(p["name"],p["price"])
```

Task 9

Store product information in a dictionary.
Example structure:

{
101: {"name":"Laptop","price":75000},

102: {"name":"Mouse","price":500}
}

```
products={p["product_id"]:{"name":p["name"],"price":p["price"]} for p in products_data}
print(products)
```

Task 10

Find the most expensive product.

```
expensive=max(products.values(),key=lambda x: x["price"])
print(expensive)
```

Task 11

Find the least expensive product.

```
cheap=min(products.values(),key=lambda x: x["price"])
print(cheap)
```

Part 3 — Orders Analysis (CSV)

Task 12

Read orders.csv .

```
import csv
orders=[]
with open("orders.csv") as f:
    reader=csv.DictReader(f)
    for row in reader:
        row["product_id"]=int(row["product_id"])
        row["quantity"]=int(row["quantity"])
        orders.append(row)
```

Task 13

Print each order.

```
for o in orders:
    print(o)
```

Task 14

Calculate the total quantity sold per product.

```
qty_per_product={}
for o in orders:
    pid=o["product_id"]
    qty_per_product[pid]=qty_per_product.get(pid, default: 0)+o["quantity"]
    print(qty_per_product)
```

Task 15

Calculate total orders per customer.
Expected structure:

{
"Rahul":3,
"Sneha":2,
"Arjun":1,
"Priya":1,
"Karan":1
}

```
orders_per_customer={}
for o in orders:
    c=o["customer"]
    orders_per_customer[c]=orders_per_customer.get(c, default: 0)+1
    print(orders_per_customer)
```

Part 4 — Sales Calculation

Using product prices and order quantities:

Task 16

Calculate revenue for each order.

```
for o in orders:
    price=products[o["product_id"]]["price"]
    revenue=price*o["quantity"]
    print(revenue)
```

Task 17

Calculate total revenue.

```
total_revenue=0
for o in orders:
    price=products[o["product_id"]]["price"]
    total_revenue+=price*o["quantity"]
print(total_revenue)
```

Task 18

Calculate total revenue per product.
Example output structure:

{
"Laptop":150000,
"Mouse":2500,
"Keyboard":4500,
"Monitor":24000
}

```
revenue_per_product={}
for o in orders:
    name=products[o["product_id"]]["name"]
    price=products[o["product_id"]]["price"]
    total=price*o["quantity"]
    revenue_per_product[name]=revenue_per_product.get(name, default: 0)+total
print(revenue_per_product)
```

Task 19

Find the highest selling product by revenue.

```
top_product=max(revenue_per_product,key=revenue_per_product.get)
print(top_product)
```

Part 5 — Customer Analysis

Task 20

Calculate total spending per customer.

```
spending={}
for o in orders:
    c=o["customer"]
    price=products[o["product_id"]]["price"]
    total=price*o["quantity"]
    spending[c]=spending.get(c, default: 0)+total
    print(spending)
```

Task 21

Find the highest spending customer.

```
top_customer=max(spending,key=spending.get)
print(top_customer)
```

Task 22

Find customers who spent more than ₹50,000.

```
high_spenders=[c for c,amt in spending.items() if amt>50000]
print(high_spenders)
```

Part 6 — Functions

Create functions for:

Task 23

Load visits from TXT.

```
def load_visits(file):  1 usage
    with open(file) as f:
        return [line.strip() for line in f]
visits=load_visits("website_vists.txt")
print(visits)
```

Task 24

Load product catalog from JSON.

```python
def load_products(file):  1 usage
    import json
    with open(file) as f:
        data=json.load(f)["products"]
        return{p["product_id"]: p for p in data}
products=load_products("products.json")
print(products)
```

Task 25

Load orders from CSV.

```python
def load_orders(file):  1 usage
    import csv
    orders=[]
    with open(file) as f:
        reader=csv.DictReader(f)
        for row in reader:
            row["product_id"]=int(row["product_id"])
            row["quantity"]=int(row["quantity"])
            orders.append(row)
    return orders
orders=load_orders("orders.csv")
print(orders)
```

Task 26

Calculate product revenue.

```python
def product_revenue(orders,products):  1 usage
    revenue={}
    for o in orders:
        name=products[o["product_id"]]["name"]
        price=products[o["product_id"]]["price"]
        total=price*o["quantity"]
        revenue[name]=revenue.get(name, default: 0)+total
        return revenue
rev=product_revenue(orders,products)
print(rev)
```

Task 27

Calculate customer spending.

```python
def customer_spending(orders,products):  1 usage
    spending={}
    for o in orders:
        customer=o["customer"]
        price=products[o["product_id"]]["price"]
        total=price*o["quantity"]
        spending[customer]=spending.get(customer, default: 0)+total
        return spending
spend=customer_spending(orders,products)
print(spend)
```

Task 28

Find top customer.

```python
def top_customer(spending):  1 usage
    return max(spending,key=spending.get)
top=top_customer(spend)
print(top)
```

Part 7 — Data Structures

Use:
list → store orders
dictionary → store product prices
set → store unique visitors
tuple → represent (product_name, revenue) pairs

```python
orders=load_orders("orders.csv")
products=load_products("products.json")
visits=load_visits("website_vists.txt")
product_prices={pid:data["price"] for pid,data in products.items()}
unique_visitors=set(visits)
revenue_per_product=product_revenue(orders,products)
product_revenue_pairs=[(name,rev) for name,rev in revenue_per_product.items()]
print(orders)
print(product_prices)
print(unique_visitors)
print(product_revenue_pairs)
```

Part 8 — Final Report Generation

Create a file called sales_report.txt.
Example output:

E-Commerce Sales Report
Total Website Visits: 10
Unique Visitors: 5
Total Revenue: 181000

Top Customer: Rahul
Product Sales
Laptop → 150000
Mouse → 2500
Keyboard → 4500
Monitor → 24000

```
with open("sales_report.txt","w") as f:
    f.write("E-Commerce Sales Report\n")
    f.write(f"Total Website Visits: {len(visits)}\n")
    f.write(f"Unique Visitors: {len(unique_visitors)}\n")
    f.write(f"Total Revenue: {total_revenue}\n\n")
    f.write(f"Top Customer: {top_customer}\n")
    f.write("Product Sales\n")
    for p,rev in revenue_per_product.items():
        f.write(f"{p} -> {rev}\n")
```

Final Challenge

Task 29

Find visitors who visited but never ordered anything.

```
visitors_no_order=unique_visitors-set(spending.keys())
print(visitors_no_order)
```

Task 30

Find customers who ordered but never visited the website more than once.

```
visit_count={}
for v in visits:
    visit_count[v]=visit_count.get(v, default: 0)+1
ordered_customers=set(spending.keys())
result=[]
for c in ordered_customers:
    if visit_count.get(c, default: 0)<=1:
        result.append(c)
print(result)
```